The Software Life Cycle Revisited

CSE 362

Software Development and Professional Practice

Winter 2023

© Royal Military College of Canada

Introduction

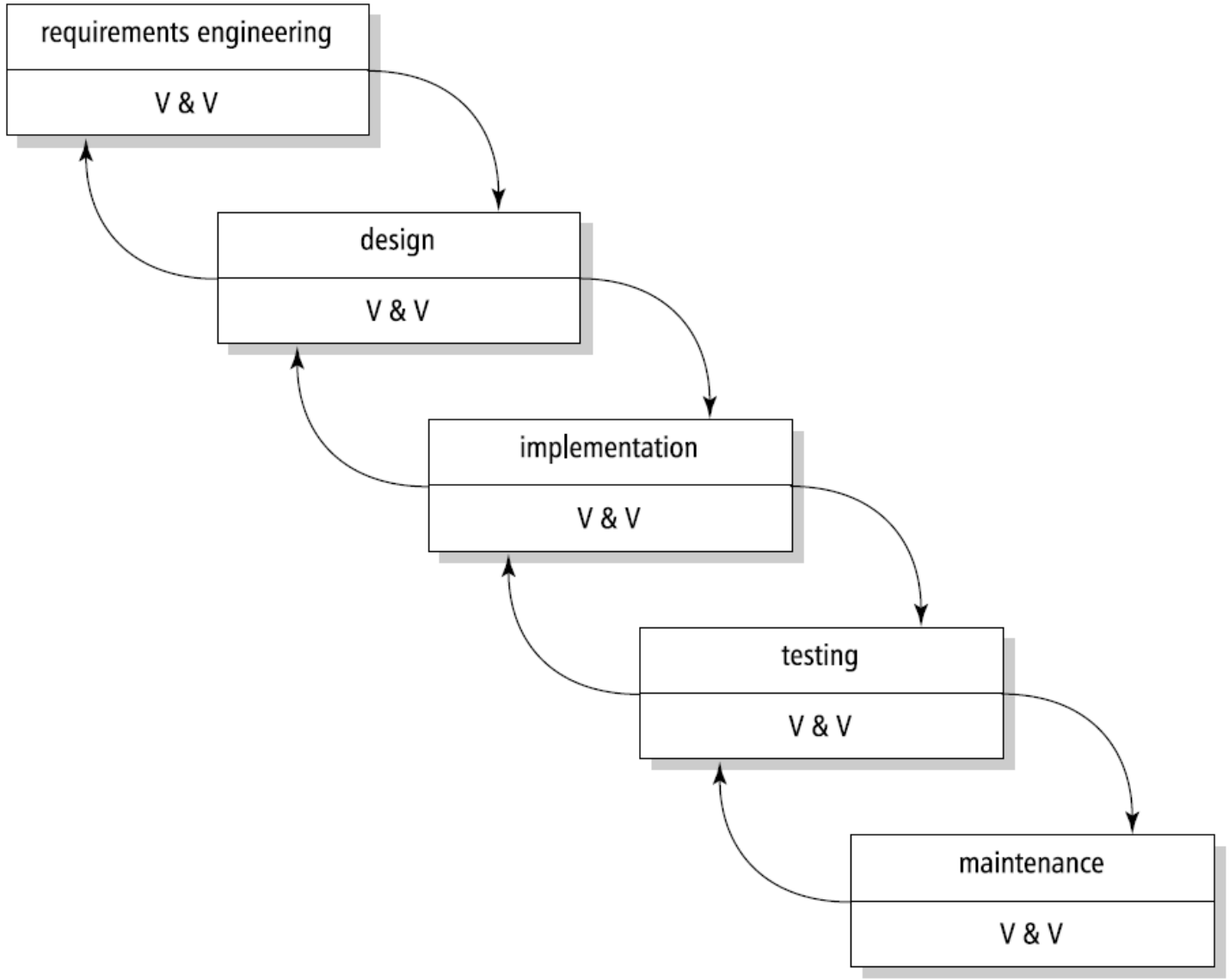
- To be able to assess progress during software development, need to use a phased approach.
  - A number of well-defined milestone events.

- Last week, we identified the SD phases:
  - requirements engineering, design, implementation, testing, maintenance.

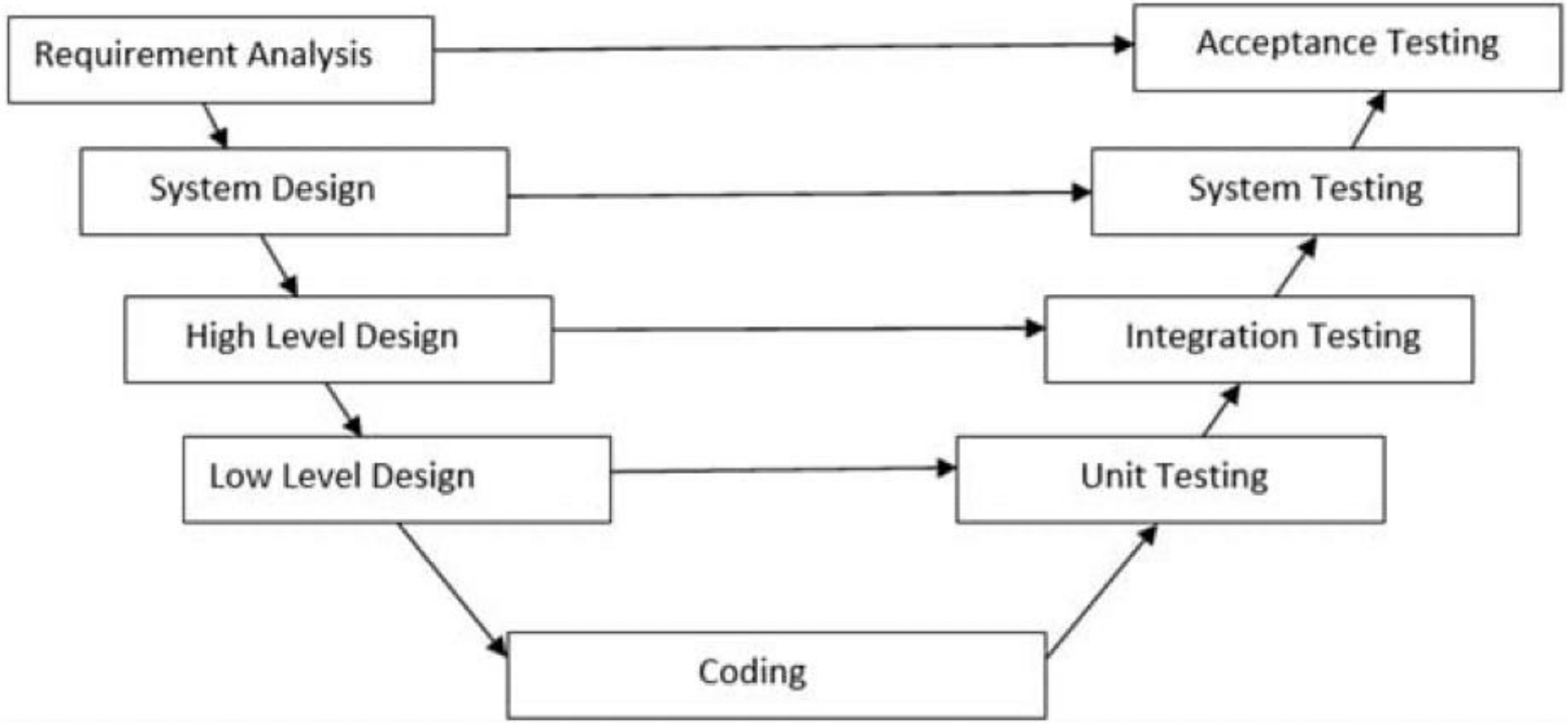
- In practice, various alternative models are used to structure the software development process.

Planning-driven
Document-driven
Heavyweight

Waterfall Model



V-shape Model



What happens in practice

In reality, the strict sequencing of phases advocated by the waterfall model is not actually obeyed.

Activity	Phase			
	Design	Coding	Integration testing	Acceptance testing
Integration testing	4.7	43.4	26.1	25.8
Coding	6.9	70.3	15.9	6.9
Design	49.2	34.1	10.3	6.4

Agile methods
Lightweight
Change-driven

Principles of Agile Software Development

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan.

Pure Agile Development

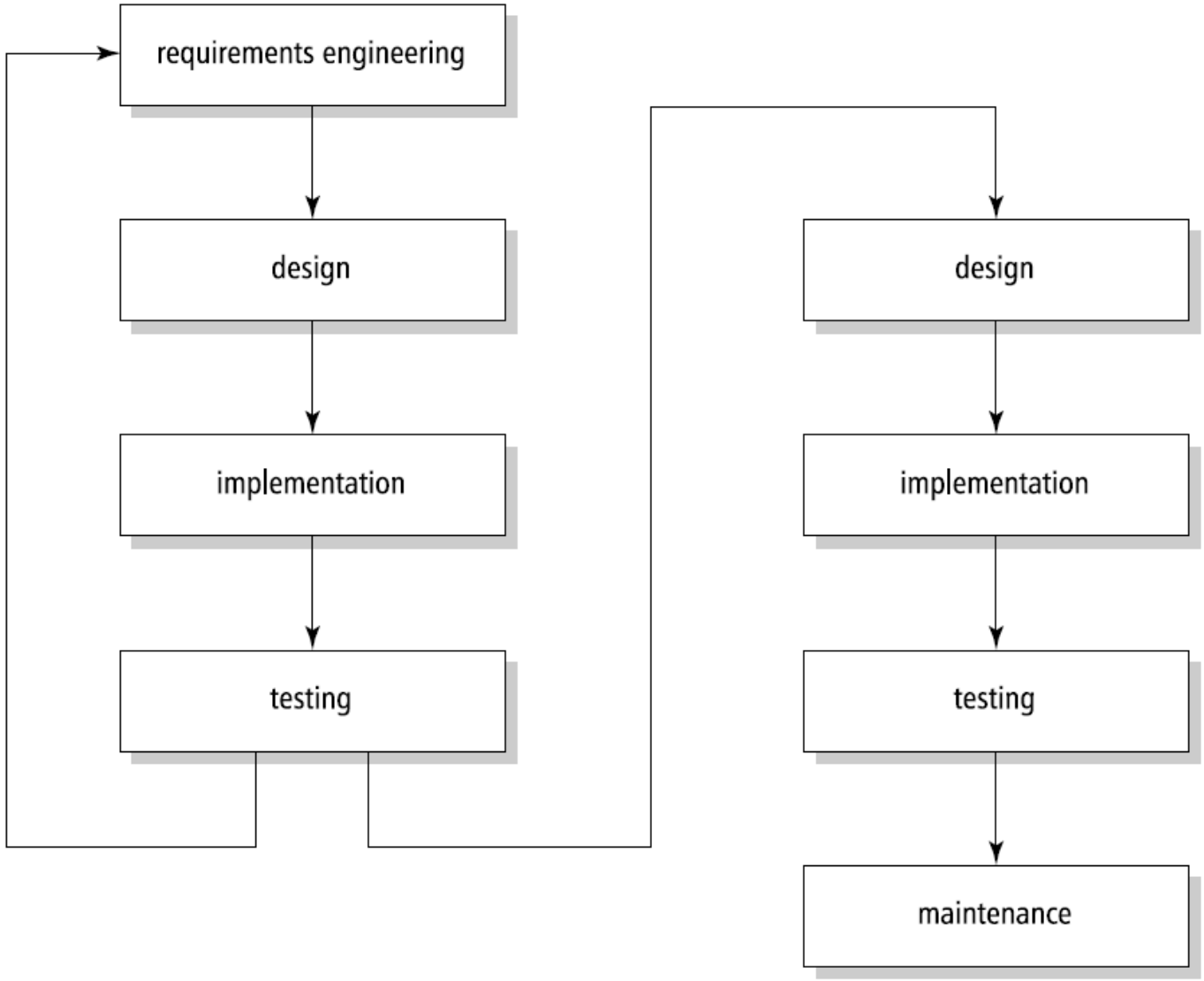
- Involve the user in every step.
- Development cycles are small and incremental.
- At the end of each cycle, the system is up and running.
- No extensive architectural or design phase up front.
- Minimal effort on documentation: it will change anyhow, why waste time?
- Team spirit and collaboration: if you have a question, don't look at documents, ask one of your pals.

Agile Development

- Some people believe that agile methods should be ‘pure’, and exhibit all of the characteristics mentioned.

- Others believe that a mixture of planning-driven and agile methods can be productive as well.
  - What could go wrong with pure Agile?

Prototyping



Prototyping: 2 types

- *Throwaway prototyping:* when the software product is not carried over from the prototyping phases to the actual production phase,

- *Evolutionary prototyping:* After a number of iterations, the user is satisfied and the last version developed is the product that goes over the production phase.

What could go well/wrong with prototyping?

- Users feel more involved in the development process and have fewer conflicts with the designers.
- Prototyping involves iterative design steps and, because of the repeated attention to the design, its quality may increase.

- The prototyping approach requires more experienced team members, if not the iterative portion becomes problematic.
- The resulting systems may be less maintainable (as a result of multiple reworks)

Incremental Development

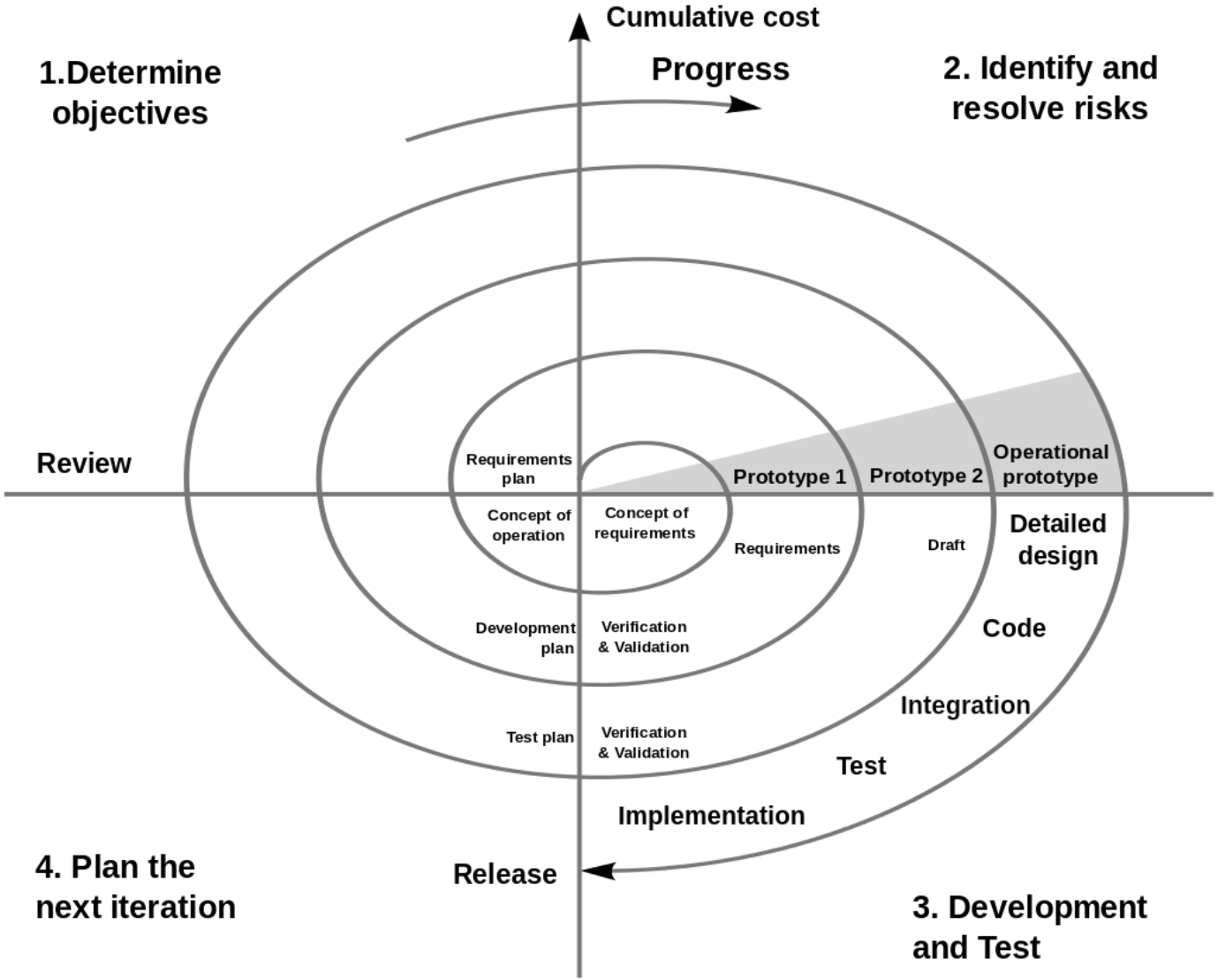
- We proceed incrementally.
- The functionality of the system is produced and delivered to the customer in *small increments*: avoids the 'Big Bang' effect.
- Instead of building software, the *software grows*.

- Help to fight the 'overfunctionality' syndrome.
- With the incremental approach, attention is first focused on the essential features.
  - Additional functionality is only included if and when it is needed

Incremental Development

- A risk-driven software development process model.
  - identify the sub-problem which has the highest associated risk;
  - find a solution for that problem;
- Adopt elements of one or more process models, such as incremental, waterfall, or prototyping.

Spiral Model



Rapid Application Development (RAD)

- has a lot in common with other iterative development process models.
- emphasizes user involvement, prototyping, reuse, the use of automated tools, and small teams
- What's different: it employs the notion of a *time box*, a fixed time frame within which activities are done.

- The RAD life cycle consists of four phases:
  - requirements planning,
  - user design,
  - construction,
  - cutover

Extreme Programming (XP)

- A pure agile method.

- XP is based on a number of best practices that have been known for long. *XP takes these practices to extreme*

- For example, it is known that code reading by your pals, such as is done in code inspections is a very effective test method.
- In XP, one does this all the time: two programmers work together behind one computer screen all the time: pair programming.

Practices of XP

XP practice	Description
The Planning Game	The scope of the next release is quickly determined. When necessary, the plan is updated
Small releases	First realize a simple system, then release next versions in short cycles
Metaphor	Use of a simple metaphor for the whole system
Simple design	Make sure the design is as simple as possible at any point in time. Remove complexity as soon as possible
Testing	Programmers continuously write unit tests, customers write acceptance tests
Refactoring	Restructure the system without changing its behaviour, to improve quality
Pair programming	All code is written by two programmers at one machine
Collective ownership	Anyone can change any code, anywhere, at any time
Continuous integration	The system is integrated and built many times a day
40-hour week	As a rule, work 40 hours a week. Working overtime should be the exception
On-site customer	Have a real user on the team, full-time
Coding standards	Establish coding standards to ease communication

Planning in XP

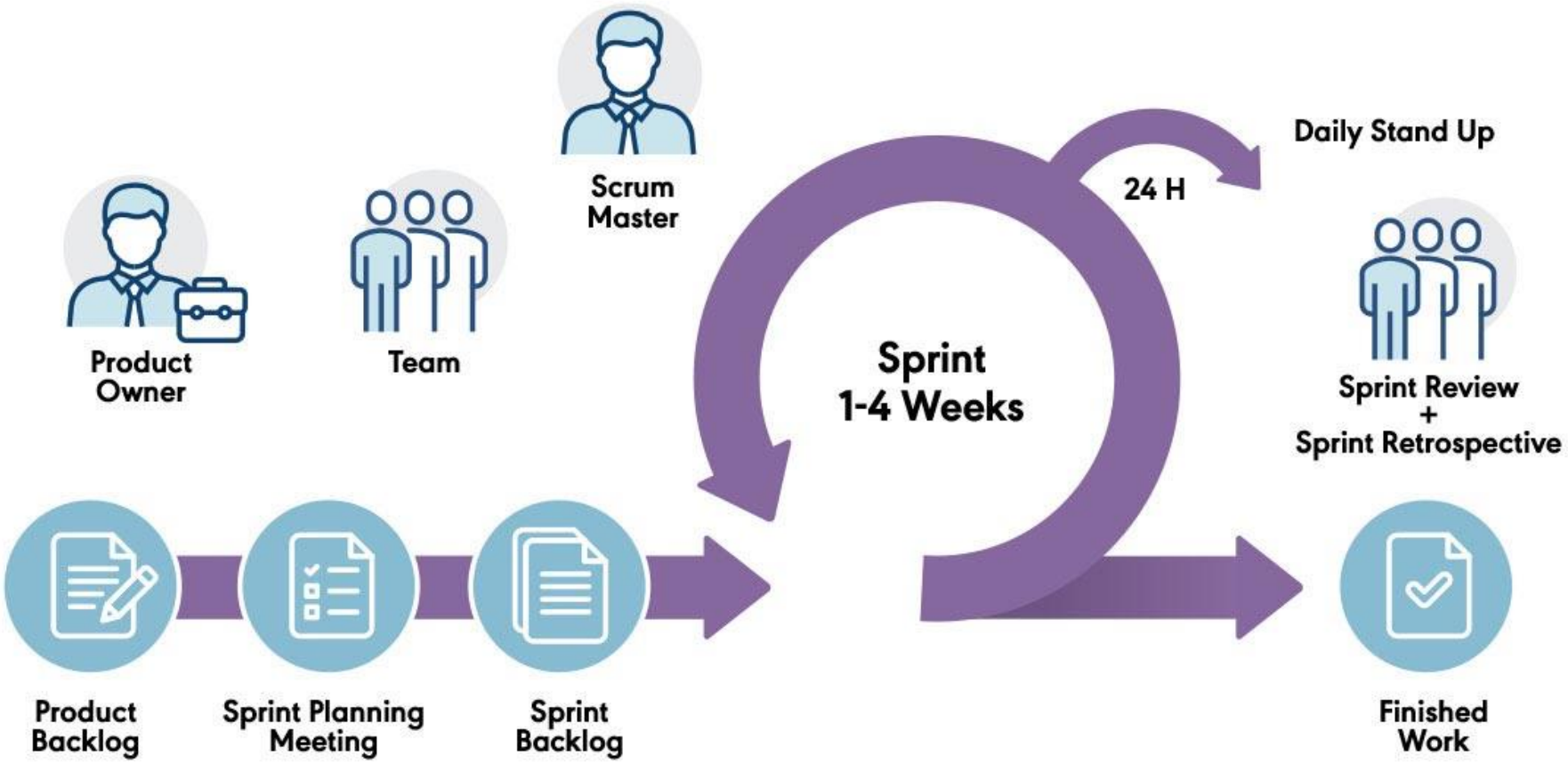
- Planning meetings are very short, and involve the immediate functionality to be delivered to the customer.
- Planning involves both the customer and the technical people.
- The customer describes desirable features of the system in stories, on index cards.
- The technical people estimate how long it takes to implement a story, decide on the order in which stories within one release will be implemented

Scrum

- Scrum uses an iterative, incremental approach when delivering products.

- Scrum is built on the theory of empiricism, i.e. *knowledge comes from experience and decisions are made based on information available at the time.*

Scrum Process



Source: https://www.pm-partners.com.au/the-agile-journey-a-scrum-overview/

Scrum Terminology - **Backlog**

- There's no shortage of terminology when working in Scrum.

- Here are some of the key terms you will come across:

- **Backlog** – a prioritised (ordered) list of everything that might be developed in a product or service and <u>the single source</u> of requirements for any changes to be made in future releases.
  - The list is typically made up of user stories, describing who wants the feature and why.

Scrum Terminology - **Sprint**

- **Sprint** – a fixed timeframe (usually two weeks) that is repeatedly used to deliver selected features from the backlog. The objective is to produce valuable product increments from each sprint.

Related to the sprint:

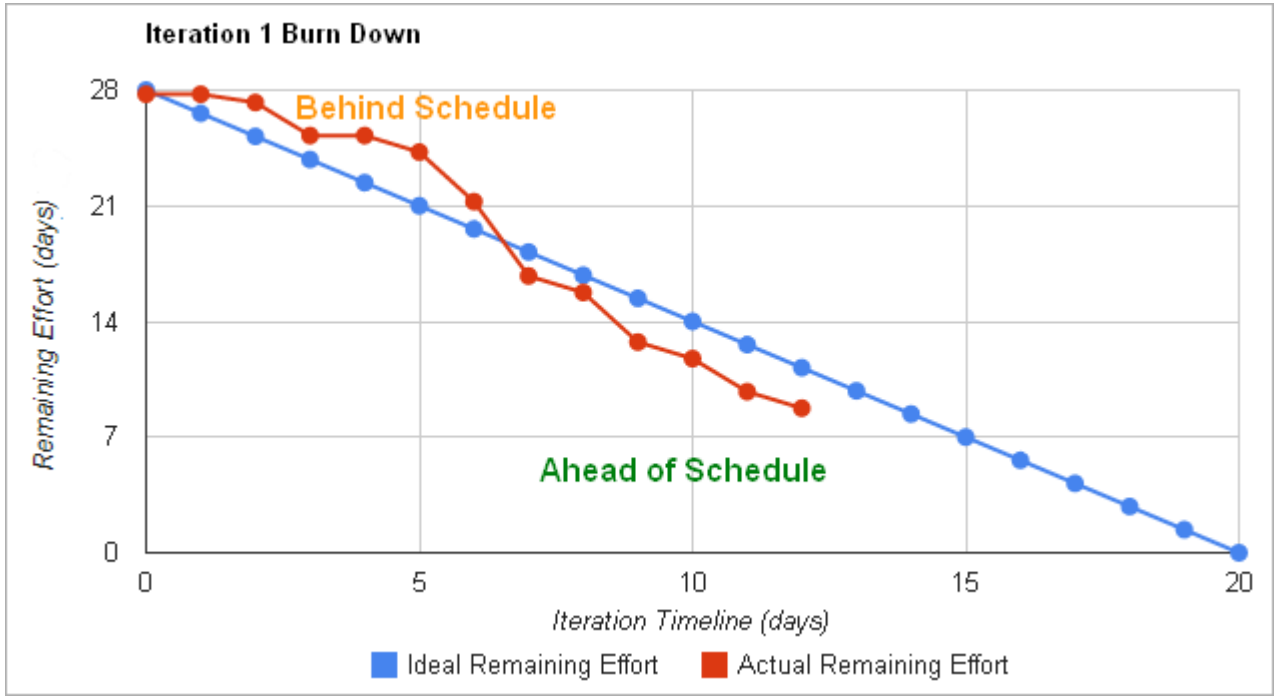
- **Sprint planning** – a collaborative event to kick off the sprint. The planning session defines the overall goal of the sprint i.e. what backlog items the team will realistically complete, and how that work will be done.

Scrum Terminology - **Daily standup**

- **Daily standup** – a short team meeting to share progress and intentions, highlight obstacles, and identify opportunities for team members to help each other get the work done.

Scrum Terminology - **Burndown chart**

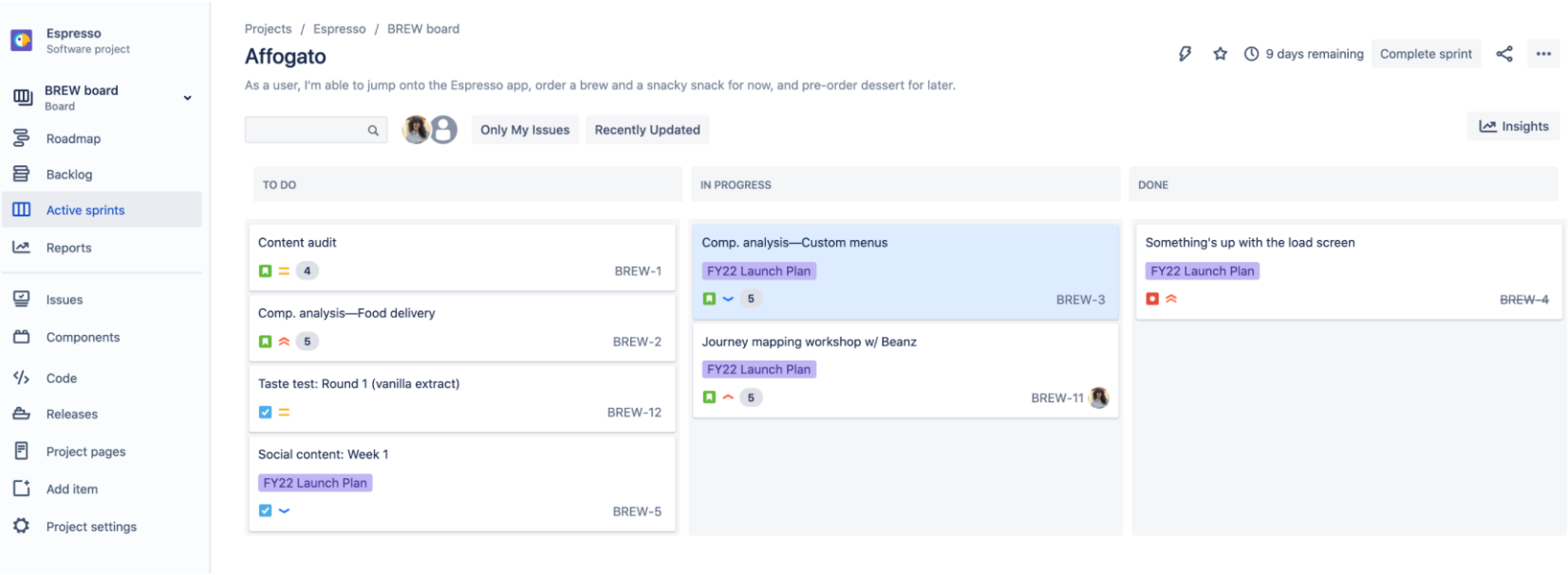
- **Burndown chart** – a technique for showing progress for a sprint, where work that is completed and work still to be done are shown with one or more lines. This is updated regularly/daily.



Source: https://dzone.com/articles/the-ideal-burn-down-chart

Scrum Terminology - **Board**

- **Visual management board** – a visual tool (physical or virtual) to display the status of each work item or user story in the sprint.
- It typically features a series of columns representing different work states and post-its representing the work, which move through the columns as work progresses.



Scrum Terminology – **User Stories**

- **User stories** – user stories are well-expressed requirements written from the perspective of an end-user outcome.

As a [stakeholder], I want to [goal] so that [motivation].

- Stakeholder – for whom are we doing this?
- Goal – what are we doing?
- Motivation – why are we doing it?

Scrum Terminology – **User Stories**, Cont.

- User stories need to be clear to everyone and verifiable. The latter means that you need to be able to later confirm that you have met the requirements of the story.

- Focus on user needs, rather than on the solution or technology domain.

Scrum Terminology – **User Stories**, Cont.

As a user, I want to be able to save my game progress so that I can re-start the game from where I left off.

As a level designer, I want to be able to use a text editor to design and edit levels for simplicity.

As a user, I want to be able to save my game in less than 2 seconds so I only have to wait a short time.

As a user, I want to be able to play this game on an Android device or on my Mac, so I can play it anywhere.

Scrum Terminology – **User Stories**, Cont.

- Keep collecting user stories until you think you have covered everything.
- Separate them into functional and non-functional (to be discussed next).
- Try to keep them from overlapping.
- Break overly large requirements up, if you can. Something overly large will be characterized by a long time-to-completion that is difficult to estimate.

Back to the Scrum Process

- The user stories will form the basis of your product backlog, expressed as issues in Jira.
- You will prioritize these and estimate time requirements (or story points).
- Select some items from the backlog to form a sprint.
- See (9 minutes): https://www.youtube.com/watch?v=XU0IIRItyFM